

End-to-End Execution Plan

Duty Doctor Roster — sequenced build order

Ordered phases to move through, each building on the last. No time budget attached — move to the next phase once the current one is genuinely done.

Strategy: Build the MVP First, Polish Second

The plan is sequenced so that a **working, deployed, rule-correct roster app** exists as early as possible.

Nice-to-have items (extra polish, edge-case toasts, styling) are pushed to the end and are the first things to cut if you fall behind.

Tier	Includes
Must-have (do not skip)	Schema + seed data, generation algorithm, persistence to Supabase, basic calendar UI, manual override that survives refresh, deploy + README
Should-have	Warn-before-overwrite on regenerate, validation toast on rule-breaking manual edit, reduced-staffing fallback logic
Nice-to-have (cut first if short on time)	Polished styling, animations, override notes/history, extensive automated tests

Build Order

Phase	What to actually do
Setup	Init repo, Node+TS project, Supabase project, .env, connect raw pg client, push empty schema file.
Schema + Seed Data	Write/extend supabase/duty-doctor-roster-schema.sql (doctors, shift_types, monthly_rosters, assignments w/ is_manual_override, leaves). Run it. Seed 6 doctors + 4 leave rows + 5 shift types.
Generation Algorithm	Pure TS function, no DB/UI dependency. Implement constraint priority order (gender → weekly off → leave → 1/day → 6/week → Rohan fixed nights → Imran caps → recovery rule → no consecutive nights → equal night/OBGYN distribution → leftover-to-Day). Unit-test with plain console asserts on a few days.
API Layer	Endpoints: POST /generate (month), GET /roster?month=, PATCH /assignment/:id (manual override), GET /leaves, POST /leaves. All raw SQL via pg. Write generated roster rows to Supabase.
React Calendar UI	Month grid, one column/row per day, shift slots per cell, doctor name + color per shift. Pull real persisted data from API. Click a slot → edit doctor / clear / mark inactive.
Manual Overrides + Validation	Save edits immediately to Supabase (is_manual_override=true). Visual badge on overridden cells. "Generate roster" button warns before overwriting non-manual cells. Toast on rule-breaking edit with an optional override-anyway + note.
Testing Pass	Walk through: a full week for Rohan (4 nights Mon–Thu), OBGYN rotation fairness, Imran's 2-night cap, a leave day, a 2-unavailable and 3-unavailable reduced-staffing day, manual override survives refresh.
README + Deploy	Write README (setup, env vars, algorithm summary, override-vs-regeneration behavior). Deploy to Vercel, confirm Supabase env vars set there too. Push repo public, verify live URL loads real data.
Final Check	Fix whatever broke on deploy. If nothing broke, use this time to double check the submission form/links.

Pre-Submission Checklist

- Schema SQL committed at `supabase/duty-doctor-roster-schema.sql`.
- Seed data includes all 6 doctors + all 4 June 2026 leave dates exactly as specified.
- No ORM anywhere in the codebase (grep for `prisma/typeorm/sequelize/drizzle` to be sure).
- No AI/LLM API calls anywhere in runtime code (grep for `openai/anthropic/gemini` imports).
- Roster is written to and read from Supabase — not hard-coded JSON in the frontend.
- Manual override persists after a hard page refresh.
- Regenerating a month does not silently wipe manually-overridden cells.
- OBGYN shift only ever assigned to Meera, Priya, or Kavya.
- Rohan never appears outside Mon–Thu for Night shifts, and never breaks the recovery rule check (he's exempt, so this should simply never block him).
- Imran never exceeds 2 Night shifts in the month, and never appears in OBGYN.
- README explains setup, env vars, algorithm, and override/regeneration interaction.
- Vercel deployment URL is live and loads the real Supabase-backed roster.
- GitHub repo is public.

If You're Running Short on Time

Cut in this order if you need to move faster — each cut still leaves a submittable, rule-correct core product:

- 1 Drop UI polish (styling, colors, animations) — a plain but functional grid is fine.
- 2 Drop the validation-toast "override with note" flow — a plain save-anyway is acceptable.
- 3 Drop automated tests — manually verify the 5–6 cases in the testing pass instead.
- 4 Drop the reduced-staffing fallback logic *last* — it's explicitly graded, so only cut it if truly out of time, and note it as a known gap in the README.

Never cut: raw SQL requirement, no-ORM requirement, no-AI-at-runtime requirement, Supabase persistence, or the manual-override-survives-refresh requirement — these are the most heavily weighted technical constraints in the brief.

Tip: commit early and often (every phase completion), so you always have a working checkpoint to deploy even if a later phase runs over.